ICT133

Proctored Online Exam - July Semester 2024

Structured Programming

Thursday, 14 November 2024 **01:00 pm - 03:00 pm**

Time allowed: 2 hours

INSTRUCTIONS TO STUDENTS:

1. This examination contains **4** questions.
2. Answer all parts of each question in the corresponding answer box.
3. All answers must be typed in the answer box and submitted in Examena.
4. This is a **Closed-book** examination.
5. You can use ONLY built-in functions and packages covered in the module, e.g., sum, max, min, math, random. You can use ONLY built-in types. Do not write classes.
6. To prevent plagiarism and collusion, your submission will be reviewed by Turnitin. The Turnitin report will only be made available to the marker and you will not be able to view it.
7. The University takes plagiarism and collusion seriously, and your Turnitin report will be examined thoroughly as part of the marking process.
8. **One (1)** A4 Sized Cheat Sheet is allowed to be brought into exam venue.

At the end of the examination.
To end the examination, please click on the "Submit and end exam" button to submit your answers.

THE UNIVERSITY RESERVES THE RIGHT NOT TO MARK YOUR SCRIPT IF YOU FAIL TO FOLLOW THESE INSTRUCTIONS.

ICT133 Copyright © 2024 Singapore University of Social Sciences (SUSS)

(Full marks: 100)

Question 1

Question 1a

Study the given *test( )* function below.

```
def test(x, y, z):
    while x <= y:
        print(x)
        x = x + z

def main():
    x = int(input("Enter start of range: "))
    y = int(input("Enter end of range: "))
    z = int(input("Enter increment value: "))
    test(x, y, z)

main()
```

What should be the values for *x*, *y*, and *z*, if the output of the program execution is:

2
4
6
8

(3 marks)

Question 1b

Produce the output of the 5 calls (line 15 to 19) to the following *testSelection( )* function.

```
1  def testSelection(x):
2      if x <= 0:
3          print("Scenario 0")
4          return
5
6      if x % 2 == 1 and x > 3:
7          print("Scenario 1")
8      elif x <= 3 or x == 7:
9          print("Scenario 2")
10     elif x > 7:
11         print("Scenario 3")
12     else:
13         print("Scenario 4")
14
15 testSelection(1)
16 testSelection(4)
17 testSelection(5)
18 testSelection(8)
19 testSelection(0)
```

(9 marks)

Question 1c

The following code segment expresses a function *secret( )* that accepts a string as parameter and will return a Boolean value.

```
1  def secret(word: str) -> bool:
2      last = word[0]
3      n = len(word)
4      for c in word:
5          if n % 2 == 0:
6              if c < last :
7                  return False
8          else:
9              if c > last:
10                 return False
11         last = c
12
13     return True
14
15 print( secret("abcd") )
16 print( secret("cbaba") )
17 print( secret("1232") )
18 print( secret("3344") )
```

Show the output of line 15 to 18.

(8 marks)

Question 2

Question 2a

Apply data structures and employ structured programming principles to develop a program.

Write a function *addTwoLists(numList1: list, numList2: list) -> list:=* that creates and returns a list that is the sum of each value in the two given lists.

- You can assume that the two parameters are list of integers.
- However, before adding the two lists, the function should check if the two lists are equal in length. If not equal, the function should return an empty list.
- For example:
    - *addTwoLists* ([1,2], [3,4]) returns [4,6]
    - *addTwoLists* ([2,3,4], [2,3,4]) returns [4,6,8]
    - *addTwoLists* ([4,4,4,4], [3,3,3]) returns [ ]

(15 marks)

Question 2b

The answers of an addition exercise are stored in a nested list as follows:

```
numList = [ [1,1,2], [2,3,4], [5,2,7], [4,1,5], [3,3,7] ]
```

In the above, 1 + 1 = 2 is correct. However, not all the additions are correct. Example, 2 + 3 = 4 is incorrect.

Write a program that determines whether each of the expressions is correct, and prints statement for each expression, and the number of correct answers for all the expressions in the list. Following is an example result of the above list:

```
1 + 1 = 2 correct.
2 + 3 = 4 incorrect.
5 + 2 = 7 correct.
4 + 1 = 5 correct.
3 + 3 = 7 incorrect.
3 correct out of 5
```

(15 marks)

Question 3

Develop an application to address practical requirements. Employ structured programming principles to develop a program, with file processing.

Designed by Samuel Morse and Alfred Vail in 1840s, the Morse code is an old system of encoding messages that is used to send telegraphic information using signals and rhythm. It uses short and long pulses - tones or lights - to represent letters and numbers.

Develop an application that allow the decoding of Morse code messages. The Morse code table are stored in “mc.txt” file and each line represent the alphabet, <space> and the corresponding Morse code. Below is an extract of the “mc.txt” file:

A .-
B -...
C -.-.
D -..
E .
F ..-.
G --.
H ....
I ..
J .---
K -.-
L .-..
M --
N -.
O ---
P .--.
Q --.-
R .-.
S ...
T -
U ..-
V ...-
W .--
X -..-
Y -.--
Z --..

Write Python functions, such that the following Python application can execute.

```
1 def main():
2     morseMapping = setupMorseMapping("mc.txt")
3     print(morseMapping)
4     morseCodes = input("Enter Morse codes (separated by space): ")
5     decodedMessage = decode(morseMapping, morseCodes)
6     print("The message is:", decodedMessage)
```

Question 3a

setupMorseMapping(filename: str) -> dict:

- The function reads the content of the given filename, populate a dictionary, with Morse code as the key and alphabet it represents.

The dictionary assigned to *morseMapping* at line 2 may look like the followings:

{'.-': 'A', '-...': 'B', '-.-.': 'C', '-..': 'D', '.': 'E', '..-.': 'F', '--.': 'G', '....': 'H', '..': 'I', '.---': 'J', '-.-': 'K', '.-..': 'L', '--': 'M', '-.': 'N', '---': 'O', '.--.': 'P', '--.-': 'Q', '.-.': 'R', '...': 'S', '-': 'T', '..-': 'U', '...-': 'V', '.--': 'W', '-..-': 'X', '-.--': 'Y', '--..': 'Z'}

(13 marks)

Question 3b

decode(morseMapping: dict, morseCodes: str) -> str:

- This function has TWO parameters: *morseMapping*, that is the dictionary populated from function *setupMorseMapping( )*, and a string morseCodes that the function must decode.
- For example, this function will decode "... --- ..." and return "S O S".
- If any part of the morse codes cannot be decoded, the function returns None.

(12 marks)

Question 4

Solve computational problems using structured programming. Employ structured programming principles to develop a program.

Read the entire question before attempting. You need to submit each part individually.

A simplified "Treasure Hunt" game can be implemented using a list.

For example, in the following list represents 10 slots, containing FIVE '1's which are the treasures, while the '0's are the empty slots.

[0, 1, 0, 1, 1, 0, 1, 0, 0, 1]

The program generates this hidden list, and the player will get 5 chances to find all the treasures.

The game rules are as follows:

1. At start of the game, create a new list of length 10, with FIVE '1's and FIVE '0's randomly placed into the list.
2. Obtain the 5 picks (indices) from player. Assume the player will enter valid indices (0-9) when playing the game. No validation is required for this part.
3. Determine the number of treasures found.
4. If all treasures are located, display a message that the player has won and the game ends.
5. If not, repeat from step 2 until player exhausted all 5 chances.
6. Display message that player has lost and show where the treasures are hidden.

<u>Sample run 1:</u>
The treasures are planted... You have 5 chances!!

Round 1: Enter your 5 picks: **1,3,5,7,9**
You got 3 hits!!
Round 2: Enter your 5 picks: **1,3,4,5,7**
You got 3 hits!!
Round 3: Enter your 5 picks: **1,3,4,9,8**
You got 5 hits!!
You are the winner!!

<u>Sample run 2:</u>
The treasures are planted... You have 5 chances!!

Round 1: Enter your 5 picks: **0,2,4,6,8**
You got 3 hits!!
Round 2: Enter your 5 picks: **0,3,4,7,8**
You got 2 hits!!
Round 3: Enter your 5 picks: **2,3,4,6,8**
You got 3 hits!!
Round 4: Enter your 5 picks: **0,2,5,6,9**
You got 2 hits!!
Round 5: Enter your 5 picks: **9,8,7,0,2**
You got 3 hits!!
You lost!!
The treasures are in: [0, 1, 1, 0, 1, 0, 0, 0, 1, 1]

Question 4a

The following 2 functions are the building blocks for the "Treasure Hunt" game:

getTreasureList( ) -> list:

- This function returns a newly shuffled list of integers, containing FIVE '0's and FIVE '1's randomly placed.
- For example: [0, 1, 0, 1, 1, 0, 1, 0, 0, 1]

countTreasures(treasureList: list, indices: list) -> int

- Given the parameter indices (a list of 5 integers), the function will use these 5 indices to locate and count the treasures found in the parameter treasureList.

Write the *getTreasureList( )* and the *countTreasures( )* functions.

(10 marks)

Question 4b

Using the 2 functions created in Q4(a), write the *main( )* function that allows a single player to play the simplified "Treasure Hunt" game.

(15 marks)

----- END OF PAPER -----